

ANTISYPHON WORKSHOP • 12 JUNE 2026

Agentic Threat Hunting Setup Guide

Please complete setup **before** the workshop. Stuck? Ask in Discord.

~10 minutes

macOS • Linux • Windows

No prior Node install needed

Free model option included

aionsec.ai — please complete setup BEFORE the workshop. Stuck? Ask in Discord.

1 Download the repo

Pick whichever is easier — both end with the project folder on your machine.

Option A — Git (recommended)

If you have `git` installed, clone it:

```
$ git clone https://github.com/faanross/antisyphon_workshop_12062026.git
$ cd antisyphon_workshop_12062026
```

Option B — Download a ZIP (no git needed)

- 1 Open github.com/faanross/antisyphon_workshop_12062026 in a browser.
- 2 Click the green **Code** button → **Download ZIP**.
- 3 Unzip it somewhere easy to find (e.g. your Desktop), then `cd` into the unzipped folder in your terminal.

2 Run the installer script

From the project folder, move into `hunting-agent/` and run the script for your OS. It is safe to re-run.

MACOS / LINUX

```
$ cd hunting-agent
$ bash ./setup.sh
```

WINDOWS (POWERSHELL)

```
> cd hunting-agent
> powershell -ExecutionPolicy Bypass -File setup.ps1
```

What the script actually does

- Checks whether you already have a supported **Node** (20.19+, 22.12+, or 24+; it deliberately avoids 21/23).
- If not, it installs **Volta** (a small, cross-platform Node version manager) and pins **Node 22.12.0** — without touching any other Node you might have.
- Runs `npm install` to download the app's dependencies.

WINDOWS NOTE The `-ExecutionPolicy Bypass` part is required — without it PowerShell refuses to run scripts. On Windows the script uses `winget` to install Volta; if you don't have winget, install Volta manually from volta.sh, reopen PowerShell, and re-run.

IF NODE WAS JUST INSTALLED Close the terminal, open a fresh one, `cd` back into `hunting-agent`, and run the script again. Volta updates your PATH, and a new terminal is the cleanest way to pick it up.

3 Choose a model provider

The labs call a real language model. You tell the app which one via a `.env` file. First, create it from the template:

```
# macOS / Linux
$ cp .env.example .env

# Windows (PowerShell)
> copy .env.example .env
```

IMPORTANT In `.env`, only ONE provider's lines should be active. Whatever provider you pick, make sure **every line for the other providers is commented out** with a leading `#` — otherwise you'll get confusing errors.

Open `.env` in any text editor. Set **one** line — `LLM_PROVIDER` — to your choice, fill in that provider's key/model, and **comment out (put a `#` in front of) every line belonging to the providers you are not using**, so that only your selected provider's lines are active. Here are all five options:

LLM_PROVIDER	WHAT YOU NEED	SET IN .ENV
<code>gemini</code>	An API key from Google AI Studio (has a free tier).	<code>GEMINI_API_KEY=...</code> <code>GEMINI_MODEL=gemini-2.5-flash</code>
<code>openai</code>	An OpenAI API key (paid account).	<code>OPENAI_API_KEY=...</code> <code>OPENAI_MODEL=gpt-5.4-mini</code>
<code>anthropic</code>	An Anthropic API key (paid account).	<code>ANTHROPIC_API_KEY=...</code> <code>ANTHROPIC_MODEL=claude-haiku-4-5-20251001</code>
<code>claude-code</code>	The Claude Code CLI already installed and logged in. No API key in <code>.env</code> .	<code>CLAUDE_CODE_MODEL=sonnet</code>
<code>codex-cli</code>	The Codex CLI already installed and logged in. No API key in <code>.env</code> .	<code>CODEX_MODEL=gpt-5.4</code>

MODEL IDS CHANGE Model names get updated over time. If a model ID is rejected, swap it for a current one from the provider's model list — the links are in the comments at the top of `.env.example`.

FREE VS PAID We're provider-agnostic — use whatever you have or prefer. Two honest notes: `gemini-2.5-flash` has a **free** tier, which is great for trying it, but the free tier is **heavily rate-limited** — during the workshop you'll likely hit `500`s and interruptions. If you'd rather work uninterrupted, a **paid** API key (any provider) is smoother: the entire course costs roughly **\$5–10 of tokens at most**, even with a stronger model such as Claude Sonnet.

Example: getting a free Gemini key (Google AI Studio)

One concrete walkthrough — the free option (keep the rate-limit caveat above in mind):

- 1 Go to `aistudio.google.com` and sign in with a Google account.
- 2 In the left sidebar open **Dashboard**, then **API Keys**.
- 3 Click **Create API key** (top right). A key is created under a project on the **Free tier**.
- 4 Copy the key.
- 5 In `.env` set:

```
LLM_PROVIDER=gemini
GEMINI_API_KEY=your-key-here
GEMINI_MODEL=gemini-2.5-flash
# keep every other provider's lines commented out with #
```

Save the file. That's it — you're ready to run.

KEEP IT SECRET Your `.env` holds a credential. Don't paste it into chat, screenshots, or commit it (the repo already ignores `.env`). Treat it like a password.

4 Start the server

From inside `hunting-agent/`:

```
$ npm run dev
```

It prints a local URL — usually `http://localhost:5173` (if that port is busy it picks the next one, so use whatever URL it prints). Open it in your browser. Leave this terminal running for the whole workshop; press `Ctrl + C` in it to stop the server.

5 Confirm it worked

A 30-second smoke test that proves the app, your key, and the model are all wired up:

- 1 On the landing page, open **Lab 01 — Our First Agent**.
- 2 Click the **Agent** tab, type anything (e.g. "What is beaconing?"), and press **Send**.
- 3 You should see a reply **stream in word by word**. That's success — the browser reached your local server, which called the model with your key. ✓

IF IT ERRORS An error here almost always means the `.env` step: provider not set, a key still left as `paste-your-key-here`, or a model ID the provider rejected. Re-check Section 3, save `.env`, and restart `npm run dev`. See Section 8 for specifics.

6

The MCP lab (Lab 05)

Every lab works with just the steps above **except** Lab 05, which makes a real call to Google Threat Intelligence over MCP. For that one lab you also need two things:

1. Install **uv** (a single binary; it fetches Python for you)

```
# macOS / Linux
$ curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
> powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

2. Add a free VirusTotal API key to **.env**

Get one at virustotal.com/gui/my-apikey, then set:

```
VT_APIKEY=your-virustotal-key
```

EXPECTED The MCP server is already bundled in the repo (**mcp-security/**) — no separate download. The **first** Lab 05 run may pause a few seconds while **uv** fetches the server's Python dependencies. That's normal, and only happens once.

7

Optional — bonus take-home

Two extras you can explore after the workshop. Neither is needed for the core labs.

Give the Agent a Shell (in-app)

The **Bonus — Give the Agent a Shell** card on the landing page runs entirely in the app you already have. It executes real commands on your machine, gated by a policy file at **hunting-agent/command-policy.json** (a **deny** / **ask** list you can edit). Read its **Lesson** and **Instructions** tabs first — and only run it on a machine you're comfortable with.

mini-distillation (command-line)

A standalone CLI that turns raw telemetry into scored candidates. It has its own dependencies (the setup script doesn't touch it), so install once:

```
# from the repo root
$ cd mini-distillation
$ npm install
$ npx tsx src/distill.ts score all examples/events.sample.json
```

It writes results to **candidates.ndjson**. See **mini-distillation/README.md** and **SCHEMA.md** for how to read the output and run it on your own data.

SYMPTOM	FIX
<code>node: command not found</code> (or "unsupported") right after setup	Close the terminal, open a new one, <code>cd</code> back in, and re-run the setup script. Volta's PATH change needs a fresh shell.
Windows: "running scripts is disabled on this system"	You omitted the policy flag. Use exactly: <code>powershell -ExecutionPolicy Bypass -File setup.ps1</code>
Windows: <code>winget</code> not found during setup	Install Volta manually from volta.sh (MSI), reopen PowerShell, re-run the script.
<code>LLM_PROVIDER</code> not set / <code>...</code> not set in <code>.env</code>	You didn't create/edit <code>.env</code> , or a value is still <code>paste-your-key-here</code> . Re-do Section 3 and restart the server.
Model errors / 401 / "invalid model"	Wrong or expired key, or a model ID the provider no longer offers. Re-check the key and pick a current model ID from the provider's list (links in <code>.env.example</code>).
"Port 5173 is in use"	Not a problem — Vite picks the next free port. Open whatever URL it prints.
<code>npm install</code> fails behind a corporate network	You're likely behind a proxy. Configure npm's proxy settings, or try from a home/non-restricted network.
Lab 05 fails or hangs	Confirm Section 6: <code>uv</code> installed and <code>VT_APIKEY</code> set. The first run is slow while <code>uv</code> fetches dependencies.

REMEMBER The detailed, per-lab instructions live [inside the app](#) — each lab opens on an **Instructions** tab. This guide only gets you to the front door.